

ECOM08A

Inteligência Artificial

Prof. João Paulo R. R. Leite
joaopaulo@unifei.edu.br

A **busca** é uma das ferramentas mais básicas e úteis na Inteligência Artificial. Ela serve de base para muitos outros algoritmos e técnicas mais avançadas. A capacidade de um **agente** (sistema de IA) resolver problemas complexos depende frequentemente de sua habilidade de **explorar de forma eficiente o espaço de soluções possíveis**.

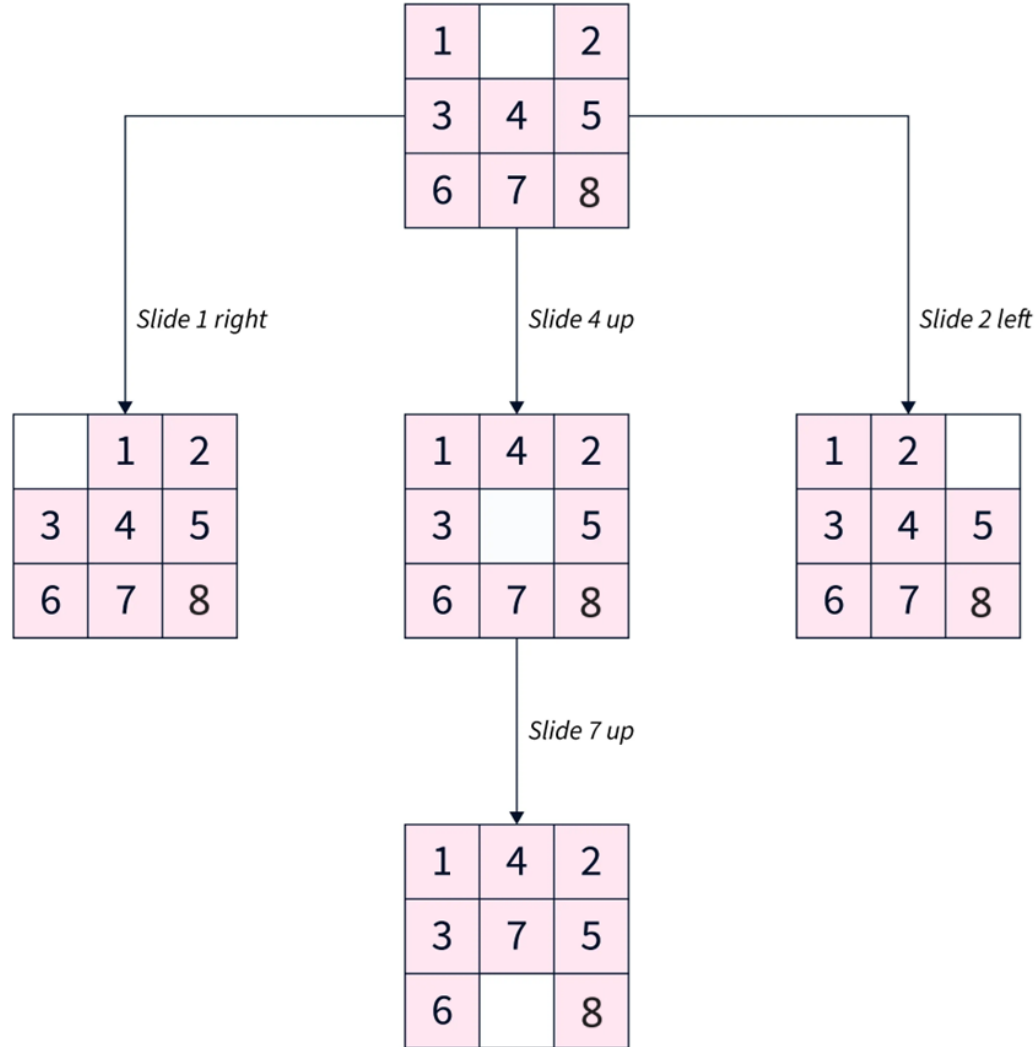
Métodos de busca são aplicáveis a uma vasta gama de problemas, desde a simples **navegação em um mapa** até problemas complexos de **otimização e raciocínio** em domínios como jogos, robótica e planejamento automático. Muitos problemas difíceis na IA, como **jogar xadrez** ou **resolver quebra-cabeças**, podem ser reduzidos a problemas de busca em espaços de estados muito grandes e complexos.

Jogos: No desenvolvimento de jogos, como **xadrez ou Go**, métodos de busca são utilizados para determinar a melhor sequência de jogadas. Algoritmos como Minimax, que também envolvem busca, são usados para prever as ações do oponente e planejar uma estratégia vencedora.

Planejamento de Rotas: Sistemas de navegação, como o **GPS**, utilizam métodos de busca para encontrar a rota mais curta ou mais eficiente entre dois pontos. Algoritmos como o Dijkstra e o A* são populares nessa aplicação.

Resolução de Quebra-Cabeças: Problemas clássicos, como o **Cubo de Rubik** ou o problema das **oito rainhas**, podem ser resolvidos por meio de métodos de busca.

Assistentes Virtuais e Chatbots: Quando um **assistente virtual** responde a uma pergunta, ele pode estar utilizando métodos de busca para consultar uma base de dados ou um gráfico de conhecimento para encontrar a resposta mais apropriada.



Como exemplo, o *8-puzzle* é um jogo de quebra-cabeça deslizante que consiste em 8 peças numeradas dispostas em uma grade 3x3, com um espaço em branco.

O objetivo do jogo é reorganizar as peças do **estado inicial** para o **estado final** objetivo, deslizando-as para o espaço em branco.

O estado inicial do quebra-cabeça corresponde à configuração inicial das peças, enquanto o estado objetivo corresponde à configuração desejada.

Os algoritmos de busca utilizam o espaço de estados para encontrar uma **sequência de ações** que transformará o estado inicial em um estado-objetivo.

O Espaço de Estados: Definição Formal

Todo problema de busca pode ser formalizado por cinco elementos

1 Estado inicial (S_0)

De onde a busca começa.

2 Ações — $A(s)$

O conjunto do que é possível fazer em cada estado.

3 Modelo de transição

$RESULT(s, a)$: para qual novo estado uma ação leva.

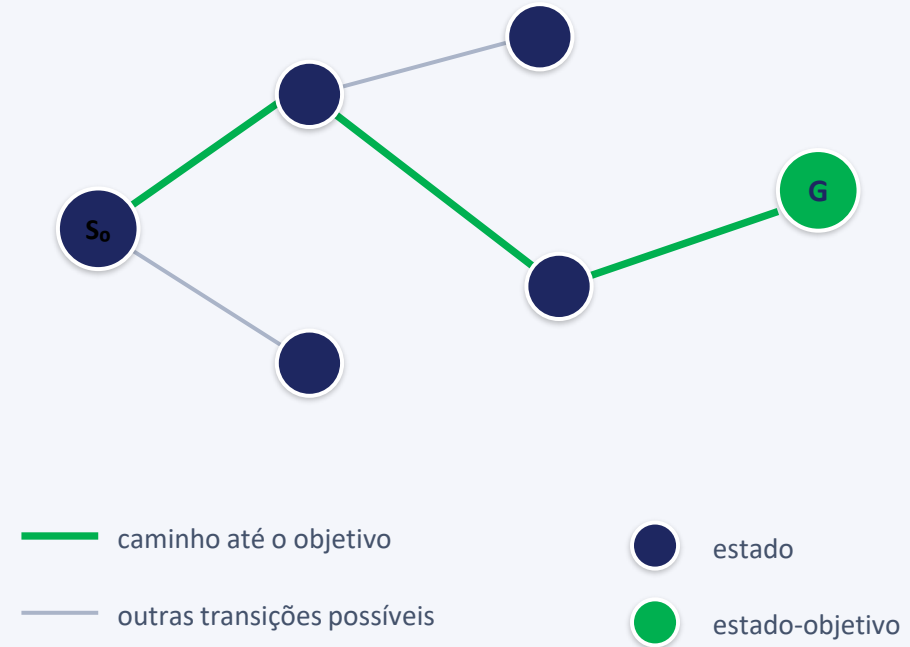
4 Teste de objetivo

Verifica se um estado é a solução procurada.

5 Custo de caminho

Soma dos custos das ações até o estado atual.

Representação como grafo



Exemplo: o Quebra-Cabeça de 8 Peças (8-Puzzle)

Cada configuração do tabuleiro é um estado; cada movimento da peça vazia é uma ação.

Estado Inicial

2	8	3
1	6	4
7		5

ações
(mover a peça vazia)



Estado Objetivo

1	2	3
8		4
7	6	5

E Estado

Disposição das 8 peças no tabuleiro 3×3.

A Ações

Mover a peça vazia: para cima, baixo, esquerda ou direita.

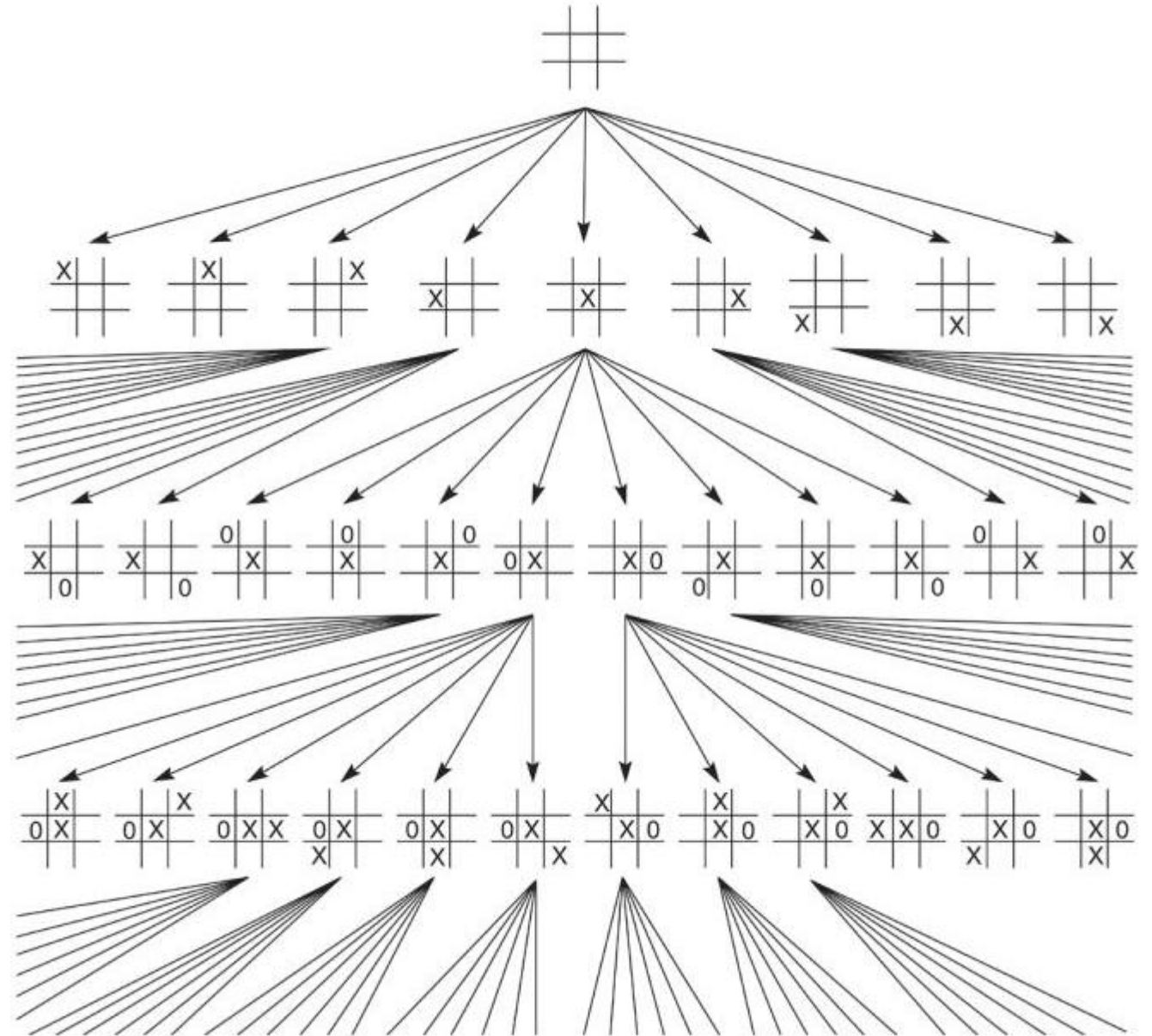
C Custo

1 unidade a cada movimento realizado.

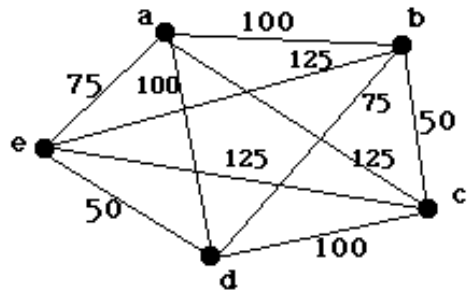
O Objetivo

Peças em ordem crescente, vazio na posição central.

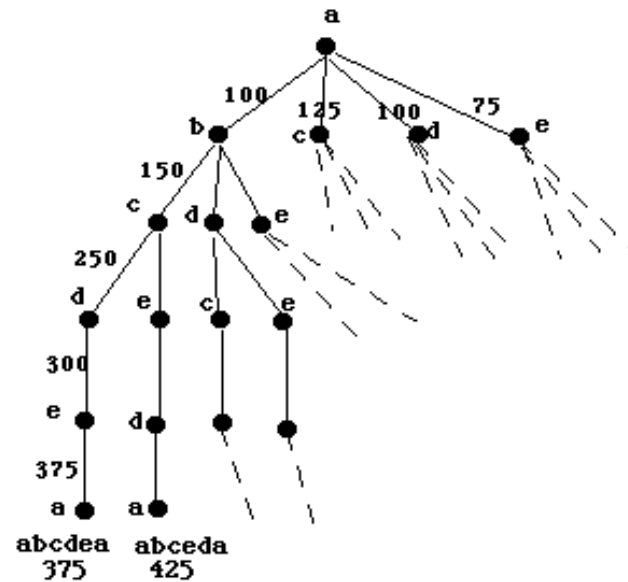
Resolver o jogo da velha também envolve modelar o problema como um **espaço de estados** e, em seguida, utilizar um **algoritmo de busca** para **explorar esse espaço** até encontrar a solução ideal (neste caso, a melhor jogada).



An Instance of the
Traveling Salesman Problem



Search Space



Caixeiro Viajante:

Começando em A, encontre o caminho mais curto que visite todas as cidades exatamente uma vez e retorne a A.

Em sua versão de força bruta, na qual é necessário **explorar cada combinação possível de caminhos**, a complexidade é muito alta.

Para N cidades, uma solução de força bruta para o problema do caixeiro viajante leva **tempo $N!$** . No entanto, há maneiras de reduzir essa complexidade. Veremos mais adiante no curso.

Tipos de Busca na IA

- **Busca Cega (não informada):** Métodos que não utilizam informações adicionais sobre o problema. Exemplos incluem a Busca em Largura (BFS) e a Busca em Profundidade (DFS). Estes algoritmos exploram o espaço de **busca de maneira sistemática**, mas podem ser ineficientes em termos de tempo e de recursos.
- **Busca Heurística (informada):** Métodos que utilizam funções heurísticas para guiar a busca de forma mais eficiente. A heurística é uma **estimativa do custo** (ou da distância) restante para alcançar a solução a partir de um estado atual. Exemplos incluem o algoritmo A* e a Busca Gulosa.

Desafios em Métodos de Busca

- **Complexidade Computacional:** À medida que o tamanho do espaço de busca aumenta, a complexidade computacional pode **crescer exponencialmente**, tornando a busca por soluções uma tarefa intensiva em termos de tempo e de memória. Com fator de ramificação b (número médio de ações por estado) e profundidade d até o objetivo, o número de nós explorados pode chegar à ordem de $O(b^d)$.
- **Espaços de Busca Infinitos (ou quase):** Em problemas em que o espaço de busca é muito grande ou potencialmente infinito, como no planejamento de longo prazo (geração de linguagem: uma frase sempre pode ter um token a mais), encontrar uma solução ótima pode ser impraticável, o que exige o uso de aproximações ou heurísticas eficazes. Vale lembrar que, às vezes, o grafo de estados é finito, mas a árvore de busca é infinita porque o algoritmo revisita os mesmos estados repetidamente (8-puzzle).
- **Escolha da Heurística:** A eficácia de um método de busca heurística depende fortemente da **qualidade da heurística** escolhida. Uma heurística mal projetada pode levar o algoritmo de busca a se comportar de forma ineficiente ou até mesmo a não encontrar a solução.

Métodos Clássicos de Busca Cega

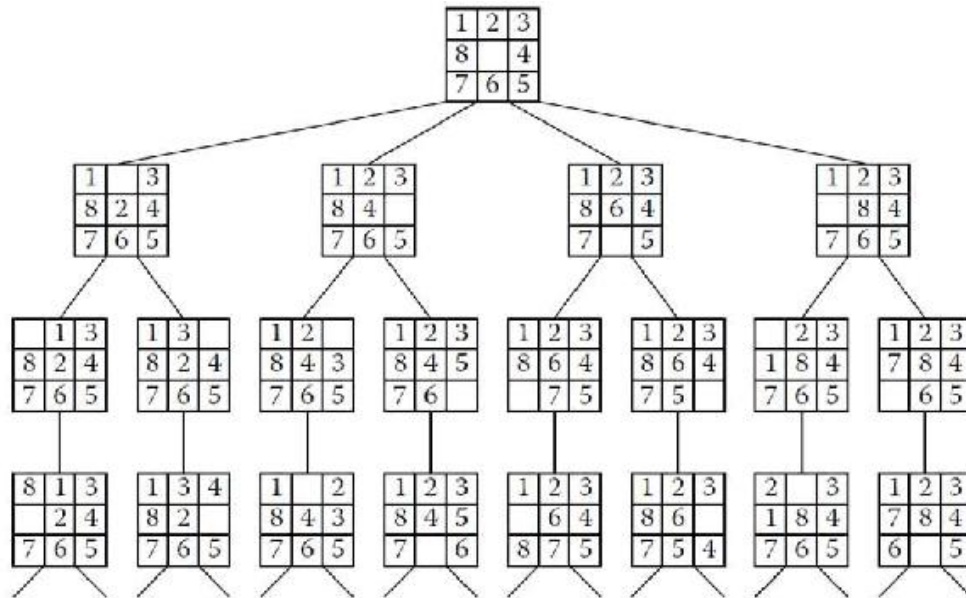
Métodos de busca cega ou não informada são algoritmos que exploram o espaço de estados de um problema sem qualquer informação adicional sobre a estrutura do problema ou a localização da solução. Em outras palavras, esses métodos não utilizam heurísticas para guiar a busca; eles **exploram o espaço de forma sistemática**, garantindo que **todos os estados possíveis sejam eventualmente considerados**.

São importantes na resolução de problemas de IA, especialmente quando **não há informações disponíveis para orientar a busca**.

Cada método tem suas próprias vantagens e desvantagens, e a escolha do método depende do tipo de problema que se pretende resolver, das **restrições de tempo e de memória e da estrutura do espaço de busca**.

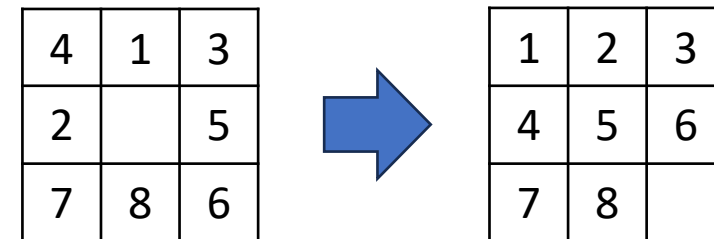
Busca em Largura

- Também chamada de **BFS** (*Breadth-First Search*).
- Explora o espaço de estados, **nível por nível**, expandindo todos os nós de um nível antes de avançar para o próximo.
- Ela utiliza uma **fila** (FIFO - *First In, First Out*) para gerenciar os nós a serem explorados. **Quem é visitado primeiro é explorado primeiro.**
- É um método **completo**, o que significa que certamente encontrará a solução se ela existir, e está garantido que encontrará a **solução mais curta** em termos de número de arestas ou “movimentos” (níveis de profundidade).



O **BFS** é adequado para resolver o *8-puzzle* porque garante encontrar a solução com o **menor número de movimentos** (todos os movimentos têm custo uniforme e o BFS explora nível a nível), é **completo** (sempre encontra uma solução se ela existir) e é **gerenciável em termos de memória e tempo**, dado o espaço de busca relativamente pequeno e finito do 8-puzzle ($181.440 \text{ estados} = 9!/2$).

Vamos partir do seguinte estado inicial:



```

# Função para imprimir o tabuleiro
def print_board(state):
    for row in state:
        print(row)
    print()

# Estado inicial do 8-puzzle
initial_state = [
    [4, 1, 3],
    [2, 0, 5],
    [7, 8, 6]
]

# Estado objetivo do 8-puzzle
goal_state = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 0]
]

# Resolver o 8-puzzle usando BFS
solution = bfs_8_puzzle(initial_state, goal_state)

# Mostrar o caminho para a solução
if solution:
    print("Solução encontrada:")
    for step in solution:
        print_board(step)
else:
    print("Nenhuma solução encontrada.")

```

Em nosso programa, o **tabuleiro** é representado por uma **matriz 3x3**, onde cada posição pode conter um número de 1 a 8 ou o espaço vazio (0).

Nessa parte do código, definimos uma função simples para impressão do tabuleiro dado um estado, e definimos os estados de interesse: inicial e objetivo.

A seguir, chamamos a função que resolve o problema utilizando busca em largura e **mostramos a solução passo a passo na tela**.

```

Solução encontrada:
[4, 1, 3]
[2, 0, 5]
[7, 8, 6]

[4, 1, 3]
[0, 2, 5]
[7, 8, 6]

[0, 1, 3]
[4, 2, 5]
[7, 8, 6]

[1, 0, 3]
[4, 2, 5]
[7, 8, 6]

[1, 2, 3]
[4, 0, 5]
[7, 8, 6]

[1, 2, 3]
[4, 5, 0]
[7, 8, 6]

[1, 2, 3]
[4, 5, 6]
[7, 8, 0]

```

```

from collections import deque

# Verifica se o estado atual eh o estado objetivo
def is_goal(state, goal):
    return state == goal

# Encontra a posicao do espaco vazio (0)
def find_zero(state):
    for i in range(3):
        for j in range(3):
            if state[i][j] == 0:
                return i, j

# Gerar novos estados a partir do estado atual
def generate_new_states(state):
    i, j = find_zero(state) # encontra o zero (i,j)
    new_states = [] # inicializa array vazio

    moves = [
        (-1, 0), # Mover para cima
        (1, 0),  # Mover para baixo
        (0, -1), # Mover para a esquerda
        (0, 1)   # Mover para a direita
    ]

    for move in moves: # para cada movimento
        new_i, new_j = i + move[0], j + move[1]
        if 0 <= new_i < 3 and 0 <= new_j < 3: # se estiver dentro do tabuleiro
            new_state = [row[:] for row in state] # pega estado atual
            new_state[i][j], new_state[new_i][new_j] = new_state[new_i][new_j], new_state[i][j] # Trocar o 0 com o numero na nova posicao
            new_states.append(new_state) # insere estado no array de novos estados

    return new_states

```

Funções Auxiliares:

- **is_goal(state, goal):** Verifica se o estado atual corresponde ao estado objetivo.
- **find_zero(state):** Localiza a posição do espaço vazio (0) no tabuleiro.
- **generate_new_states(state):** Gera todos os estados possíveis movendo o espaço vazio para cima, para baixo, para a esquerda ou para a direita.

```

# BFS para resolver o 8-puzzle
def bfs_8_puzzle(initial_state, goal_state):
    queue = deque([(initial_state, [])]) # coloca estado inicial na fila em forma de tupla (estado atual, caminho)
    visited = set() # nenhum no ainda foi visitado

    while queue: # enquanto a fila nao estiver vazia
        state, path = queue.popleft()

        # Verificar se o estado atual eh o objetivo
        if is_goal(state, goal_state):
            return path + [state] # retorna caminho ate o estado atual

        # Converter o estado para um hashable (tupla) para colocar no "set" (conjunto) de visitados
        state_tuple = tuple(tuple(row) for row in state)
        if state_tuple in visited:
            continue

        visited.add(state_tuple)

        # Gerar e adicionar novos estados a fila
        for new_state in generate_new_states(state):
            queue.append((new_state, path + [state]))

    print("Nenhuma solucao encontrada.")
    return None

```

A lista de listas (matriz) não é Hashable e não pode ser colocada em um set. Por isso, precisamos transformá-la em uma tupla de tuplas. Tuplas são imutáveis, listas não.

A função implementa a **BFS** para **explorar todos os estados possíveis do quebra-cabeça 8-puzzle** a partir do estado inicial até o estado objetivo.

A BFS expande os estados ao adicionar cada possível movimento à fila, garantindo que todos os estados possíveis sejam explorados. A busca para quando o estado objetivo é encontrado.

Utilizamos uma fila (deque) para gerenciar os estados a serem explorados. Um conjunto (visited) é utilizado para rastrear estados já visitados e evitar ciclos.

- **Vantagens**

- **Completo:** Se existe uma solução, a BFS irá encontrá-la.
- **Ótimo:** Para problemas onde o custo de cada movimento é o mesmo, a BFS encontra a solução com o menor número de movimentos (ou de profundidade mínima).

- **Desvantagens**

- **Consumo de Memória:** A BFS pode consumir muita memória, pois precisa armazenar todos os nós em um nível antes de avançar.
- **Ineficiência em Espaços de Busca Grandes:** Em espaços de busca muito grandes, a BFS pode se tornar impraticável devido à explosão combinatória.

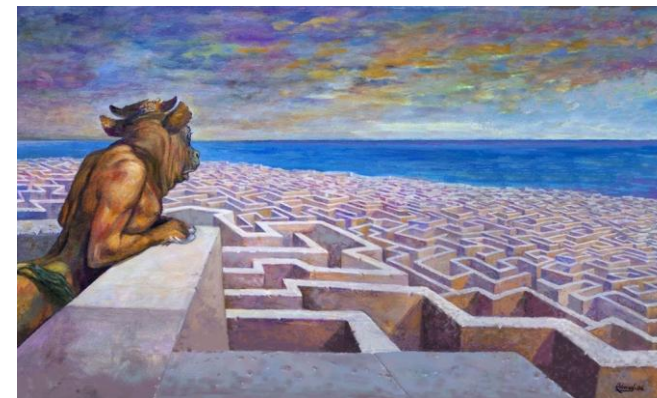
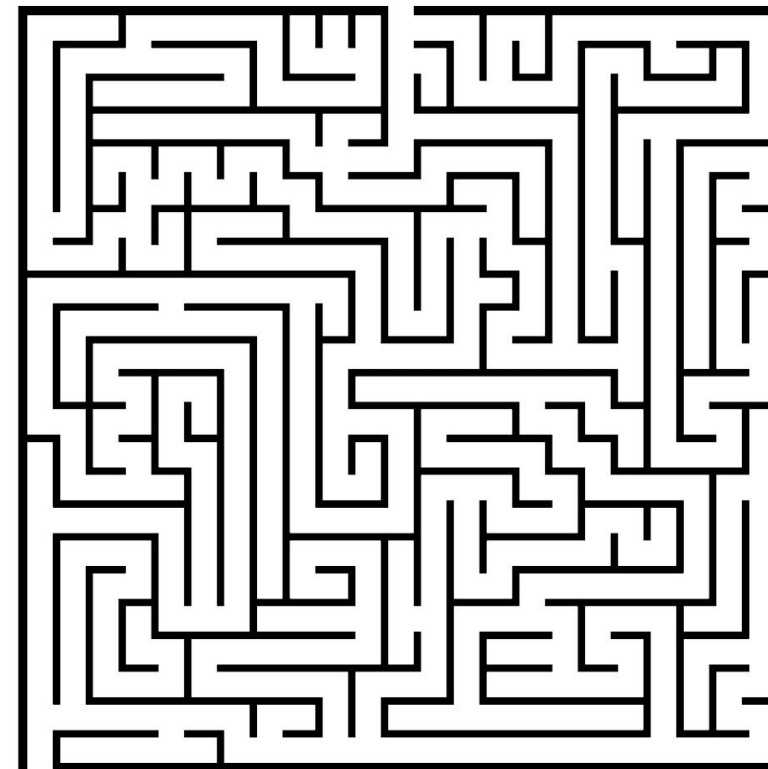
Busca em Profundidade

- Também chamada de **DFS** (*Depth-First Search*).
- A busca em profundidade explora o espaço de estados **o mais profundamente possível** antes de retroceder (*backtrack*) para explorar outras alternativas.
- Ela utiliza uma **pilha** (LIFO - *Last In, First Out*) para gerenciar os nós a serem explorados. Significa que o último nó visitado é o primeiro que iremos explorar a seguir. Isso dá a “profundidade” do algoritmo.
- Diferentemente da BFS, a DFS pode ficar presa em caminhos profundos sem solução, especialmente em espaços de busca infinitos.

O **DFS** é uma escolha adequada para resolver o problema da **busca pela saída de um labirinto** devido à sua capacidade de explorar completamente os caminhos, à simplicidade de implementação e à robustez em cenários com muitos caminhos possíveis e poucos recursos disponíveis para manter uma memória grande.

O DFS explora cada caminho completamente até chegar a um **beco sem saída (ou à própria solução)** antes de retroceder e tentar um caminho alternativo. Essa característica permite que o algoritmo siga um caminho o mais longe possível, explorando todas as opções antes de desistir desse caminho e tentar outro.

Ele é particularmente útil em **labirintos**, onde o objetivo é encontrar qualquer caminho para a saída, mas **não necessariamente o mais curto**.



```

# Exemplo de labirinto
maze = [
    [1, 1, 1, 1, 1, 1, 1],
    [1, 'S', 0, 0, 1, 0, 1],
    [1, 0, 1, 0, 1, 0, 1],
    [1, 0, 1, 0, 0, 0, 1],
    [1, 0, 1, 1, 1, 1, 1],
    [1, 0, 0, 0, 0, 0, 'E'],
    [1, 1, 1, 1, 1, 1, 1]
]

# Definir os pontos de inicio e fim
start = (1, 1) # Posicao de 'S'
end = (5, 6)   # Posicao de 'E'

# Resolver o labirinto usando DFS
path = dfs_maze(maze, start, end)

if path:
    print("Caminho encontrado:")
    print_path(maze, path)
else:
    print("Nenhum caminho encontrado.")

```

Em nossa implementação, o labirinto será representado como uma **matriz de duas dimensões** (lista de listas), onde:

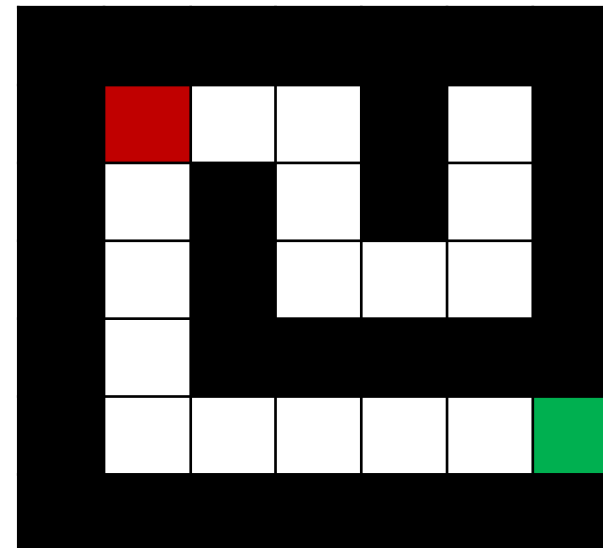
0 representa um caminho livre.

1 representa uma parede.

S representa o ponto de início (*Start*).

E representa o ponto final (*End*).

No exemplo ao lado, temos o seguinte:



```

Caminho encontrado:
1 1 1 1 1 1 1
1 * 0 0 1 0 1
1 * 1 0 1 0 1
1 * 1 0 0 0 1
1 * 1 1 1 1 1
1 * * * * *
1 1 1 1 1 1 1

```

```
def dfs_maze(maze, start, end):
    rows, cols = len(maze), len(maze[0]) # dimensoes do labirinto
    stack = [(start, [start])] # Pilha contendo o no inicial e o caminho percorrido ate ele
    visited = set() # Conjunto para rastrear os nos que ja foram visitados

    while stack: # enquanto a pilha nao ficar vazia
        (current_pos, path) = stack.pop() # pega prpximo no a ser explorado
        if current_pos == end: # se for o destino, retorna o caminho (novelo de linha)
            return path

        visited.add(current_pos) # marca como visitado (marca com giz)
        row, col = current_pos # pega sua posicao atual

        # Movimentos possiveis: cima, baixo, esquerda, direita
        possible_moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]

        for move in possible_moves: # para cada movimento possivel
            next_row, next_col = row + move[0], col + move[1]
            # se estiver nos limites do labirinto e nao for parede
            if 0 <= next_row < rows and 0 <= next_col < cols and maze[next_row][next_col] != 1:
                next_pos = (next_row, next_col) #pega proxima posicao e
                if next_pos not in visited: # se nao foi visitada ainda
                    stack.append((next_pos, path + [next_pos])) # empilha

    return None # Se nao houver caminho

# Imprime o caminho encontrado no labirinto
def print_path(maze, path):
    for row, col in path: # se estiver no caminho, imprime *
        maze[row][col] = "*"
    for row in maze: # senao, imprime conteudo da celula
        print(" ".join(str(cell) for cell in row))
```

A função recebe a matriz que representa o labirinto (***maze***) e as coordenadas de início e fim. Em sua implementação, ela utiliza uma **pilha** (*stack*) para gerenciar os nós (posições) a serem explorados, armazenando nela também o caminho percorrido até cada nó. Além disso, utiliza um **conjunto** (*set*) para armazenar os nós que já foram visitados, **evitando ciclos**.

O algoritmo tenta todos os caminhos possíveis a partir do ponto de partida usando DFS até encontrar o ponto final. **Ele faz isso expandindo cada caminho o mais profundamente possível antes de retroceder.**

São considerados quatro possíveis movimentos: **cima, baixo, esquerda, e direita**. Cada movimento é verificado para garantir que não ultrapasse os limites do labirinto e que não seja uma parede (1). Se o movimento for válido e leve a uma posição que ainda não foi visitada, ele é adicionado à pilha para exploração futura.

- **Vantagens**

- **Consumo de Memória:** A DFS requer menos memória do que a BFS, pois armazena apenas a pilha de nós em expansão.
- **Simplicidade:** A DFS é simples de implementar e pode ser útil em problemas em que a solução está em profundidades maiores e o espaço de busca não é muito grande.

- **Desvantagens**

- **Não Completa:** A DFS pode não encontrar uma solução se o espaço de busca for infinito ou se a solução estiver em uma ramificação muito profunda.
- **Não Ótimo:** A DFS não garante encontrar a solução de menor custo ou profundidade mínima.

Como resolver o problema da busca infinita?

Utilizando a **Busca em Profundidade Limitada (DLS**, de *Depth-Limited Search*) - uma variação da DFS que introduz um limite na profundidade que o algoritmo pode alcançar.

Para este tipo de busca, a **escolha do limite é crucial**. Um limite baixo pode impedir que a solução seja encontrada, enquanto um limite alto pode não resolver o **problema de consumo de memória**.

Suas vantagens são que ela:

- Permite evitar que o algoritmo fique preso em ramificações infinitas, e;
- Se você souber um limite razoável para a profundidade onde a solução pode ser encontrada, a DLS pode ser eficiente.

Busca Bidirecional

- Também chamada de *Bidirectional Search*.
- É uma técnica que executa **duas buscas simultâneas**: uma a partir do estado inicial e outra a partir do estado objetivo. Quando essas duas buscas se encontram, a solução é encontrada.
- Este método é frequentemente utilizado quando o **estado inicial e o objetivo são conhecidos**.
- Reduz o espaço de busca ao explorar a metade do caminho de cada lado, tornando o processo potencialmente mais rápido. No entanto, pode ser mais **complexa de implementar** devido à necessidade de gerenciar duas buscas e de **detectar a interseção entre elas**.

Um exemplo clássico de um problema que pode ser eficientemente resolvido usando busca bidirecional é o **problema de encontrar o caminho mais curto entre dois nós em um grafo não ponderado**. A busca bidirecional é particularmente eficiente porque, em vez de explorar o espaço de busca a partir de um único ponto, ela explora simultaneamente a partir do ponto de partida e do ponto de destino até que as duas buscas se encontrem.

Vamos dar uma olhada no **código**:

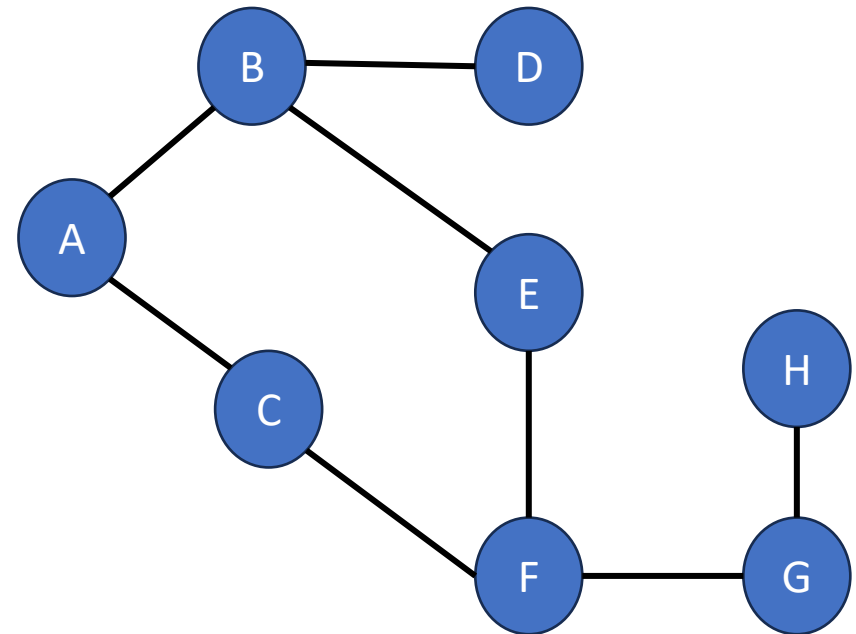
- Dado um grafo não ponderado, encontre o caminho mais curto entre dois nós, *start* e *goal*.
- O grafo é representado por uma lista de adjacência, onde cada nó tem uma lista de vizinhos diretamente conectados.

```
# Exemplo de grafo nao ponderado representado por lista de adjacencia
graph = {
    'A': ['B', 'C'],
    'B': ['A', 'D', 'E'],
    'C': ['A', 'F'],
    'D': ['B'],
    'E': ['B', 'F'],
    'F': ['C', 'E', 'G'],
    'G': ['F', 'H'],
    'H': ['G']
}

start = 'A'
goal = 'H'

# Encontra o caminho mais curto usando busca bidirecional
shortest_path = bidirectional_bfs(graph, start, goal)

if shortest_path:
    print("Caminho mais curto encontrado:", " -> ".join(shortest_path))
else:
    print("Nenhum caminho encontrado entre", start, "e", goal)
```

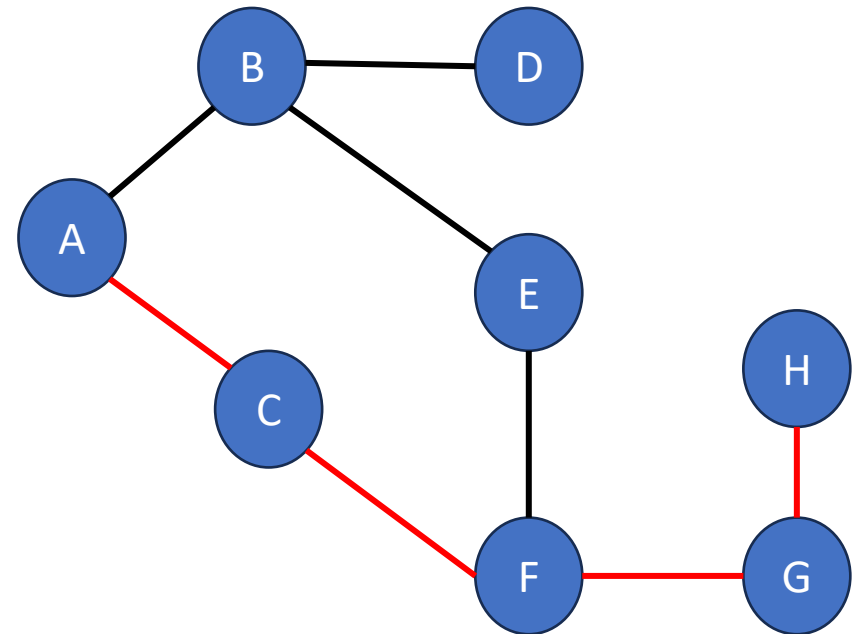


```
# Exemplo de grafo nao ponderado representado por lista de adjacencia
graph = {
    'A': ['B', 'C'],
    'B': ['A', 'D', 'E'],
    'C': ['A', 'F'],
    'D': ['B'],
    'E': ['B', 'F'],
    'F': ['C', 'E', 'G'],
    'G': ['F', 'H'],
    'H': ['G']
}

start = 'A'
goal = 'H'

# Encontra o caminho mais curto usando busca bidirecional
shortest_path = bidirectional_bfs(graph, start, goal)

if shortest_path:
    print("Caminho mais curto encontrado:", " -> ".join(shortest_path))
else:
    print("Nenhum caminho encontrado entre", start, "e", goal)
```



Caminho mais curto encontrado: A -> C -> F -> G -> H

```
def bidirectional_bfs(graph, start, goal):
    if start == goal: # terminou a busca!
        return [start]

    # Inicializar duas filas: uma começando de 'start' e outra de 'goal'
    queue_start = deque([(start, [start])])
    queue_goal = deque([(goal, [goal])])

    # Conjuntos para rastrear no's visitados a partir de cada extremidade
    visited_start = {start: [start]}
    visited_goal = {goal: [goal]}

    while queue_start and queue_goal: # enquanto as duas filas nao estiverem vazias
        # Expandir a busca a partir do lado do 'start'
        path = bfs_step(queue_start, visited_start, visited_goal)
        if path:
            return path

        # Expandir a busca a partir do lado do 'goal'
        path = bfs_step(queue_goal, visited_goal, visited_start)
        if path:
            return path

    return None # Se nao houver caminho
```

A função **bfs_step** realiza uma única etapa de BFS a partir de um dos lados. Ela **explora os vizinhos do nó atual** e **verifica se algum já foi visitado pela busca oposta**.

Se as duas buscas se encontram, o caminho é reconstruído **juntando os caminhos** encontrados de ambas as extremidades.

Em grafos não ponderados, o BFS **sempre encontrará o caminho mais curto** (em termos de número de arestas) entre dois nós porque explora primeiro todos os caminhos com uma única aresta, depois com duas arestas, e assim por diante. **Por isso usamos BFS e não DFS.**

O algoritmo vai **expandindo passo a passo** (*bfs_step*), **alternando o lado** da busca (*start* e *goal*), até que os caminhos se encontrem e a variável *path* não seja igual a *None* (semelhante a *null* em C++).

```
def bfs_step(queue, visited_from_this_side, visited_from_other_side):
    current_node, path = queue.popleft()

    # Explorar os vizinhos do no atual
    for neighbor in graph[current_node]:
        if neighbor in visited_from_this_side:
            continue

        new_path = path + [neighbor]
        queue.append((neighbor, new_path))
        visited_from_this_side[neighbor] = new_path

    # Se encontrarmos o no no lado oposto da busca, unimos os caminhos
    if neighbor in visited_from_other_side:
        return path + visited_from_other_side[neighbor][::-1]
        # [::-1] inverte a lista, não uso new_path para não suplicar a "emenda"

    return None
```

- **Vantagens**

- **Eficiência:** A busca bidirecional é mais eficiente em termos de tempo e espaço porque reduz a quantidade de nós explorados. Em um grafo de grande escala, em vez de explorar todos os nós a partir de um único ponto, ela explora menos nós de ambas as extremidades, o que pode reduzir o tempo de execução exponencialmente.
- **Aplicabilidade:** Essa técnica é útil em muitos problemas onde tanto o ponto de partida quanto o ponto de chegada são conhecidos, como na navegação por mapas, em sistemas de roteamento, e em jogos de tabuleiro.

- **Desvantagens**

- **Complexidade de Implementação:** Pode ser mais complexo de implementar devido à necessidade de gerenciar duas buscas e de detectar a interseção entre elas.
- **Necessidade de Estado Objetivo Definido:** Funciona bem quando o estado objetivo é claramente definido e conhecido.

Mão na Massa!

Faça a busca bidirecional, passo a passo, no grafo ao lado e entregue em uma folha de papel.

